11

Real-Time Collaborative Virtual Whiteboard

Software Engineering Mini Project — Team 11

Rachit S Mane · P Chinni Krishna Reddy · Pranav V Menon · Prithvi W

Final evaluation · November 2026

The problem we were given

Build a web whiteboard on which several people can draw, write and edit at the same time, see every change live, and go back to earlier versions.

1

Ordering

Concurrent edits from many browsers must end in one identical board for everyone.

2

Recovery

Connections drop and servers restart; acknowledged work must never be lost.

3

Access

A shared link must never give more rights than intended, and history must stay intact.

Objectives

1

Collaborate live

Every accepted change reaches all participants without a reload, in the same order.

2

Lose nothing confirmed

Store a change before confirming it; resynchronise after any disconnection.

3

Share safely

Owner, Editor and Viewer roles enforced on the server; expiring, revocable links.

4

Keep history

Automatic versions, read-only preview and an Owner-only restore that never deletes history.

Our solution



A browser whiteboard backed by one authoritative server

- React client draws instantly and syncs every change over Socket.IO
- Node.js server orders, validates and stores each change in MongoDB before confirming it
- REST API for accounts, boards, sharing and version history

Key features

Drawing tools

Pen, rectangle, ellipse, line, text, select/move/restyle, eraser, clear

Real-time sync

Server-ordered operations, optimistic UI, live participant list

Sharing & roles

Owner / Editor / Viewer, expiring links, guests, ownership transfer

Version history

Automatic versions, preview, Owner-only restore

Reconnection

Offline banner, disabled tools, automatic resync with the server

Security

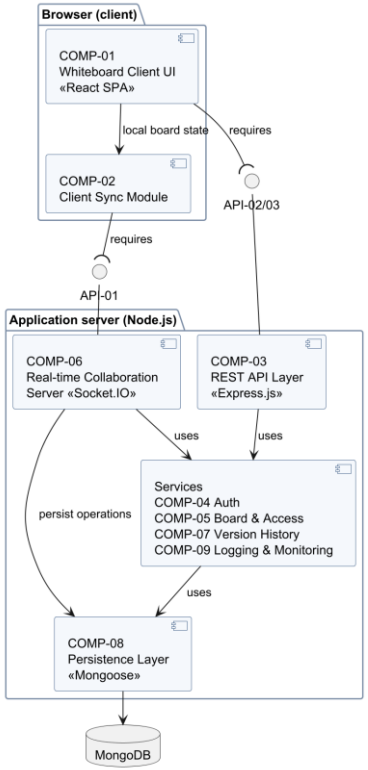
bcrypt, JWT, one permission policy, schema validation, rate limits

Also built (outside the SRS v1.0 baseline, scope decision pending): PNG export and board rename.

Technology stack

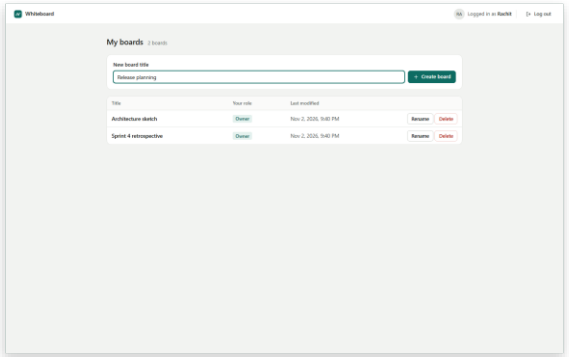
Client	React 18 · Vite 6 · HTML Canvas 2D · Socket.IO client
Server	Node.js · Express 5 · Socket.IO 4 · Ajv · JWT · bcrypt
Database	MongoDB 8 · Mongoose 8
Shared	@wb/shared: sync module, object model, error codes
Testing	Jest · Supertest · scripted Socket.IO clients · Playwright
Delivery	GitHub Issues & milestones · Pull Requests · GitHub Actions CI

System architecture

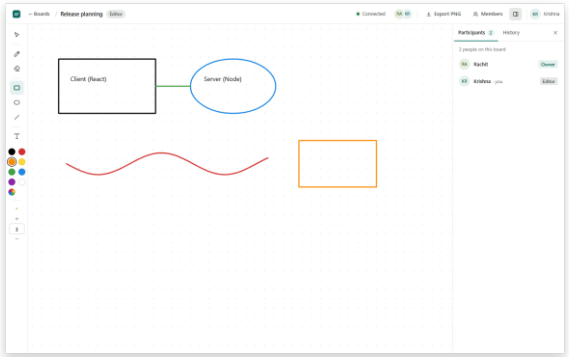


1. **Browser**
   COMP-01 UI · COMP-02 Sync module
2. **API-01**
   Socket.IO /board: operations, presence, notifications
3. **API-02/03**
   REST /api/v1: accounts, boards, links, members, versions
4. **Server**
   COMP-06 real-time · COMP-03 REST · COMP-04/05/07 services · COMP-09 logging
5. **Storage**
   COMP-08 Mongoose models → MongoDB

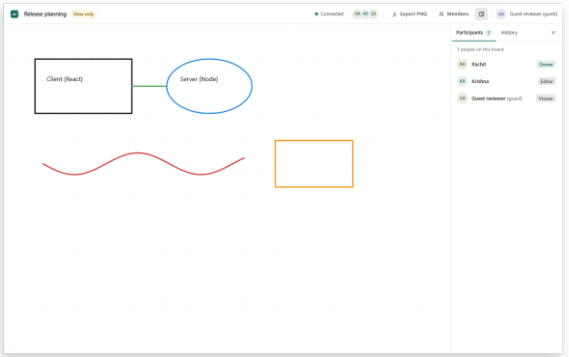
Frontend and user interface



Dashboard



Editor view



Guest Viewer (read-only)

- BoardPage.jsx owns the session: socket, sync client, roles, dialogs
- Canvas.jsx turns pointer input into board objects; render.js draws them
- Role-aware UI: tools hidden for Viewers — the server enforces the rules anyway

Backend and API

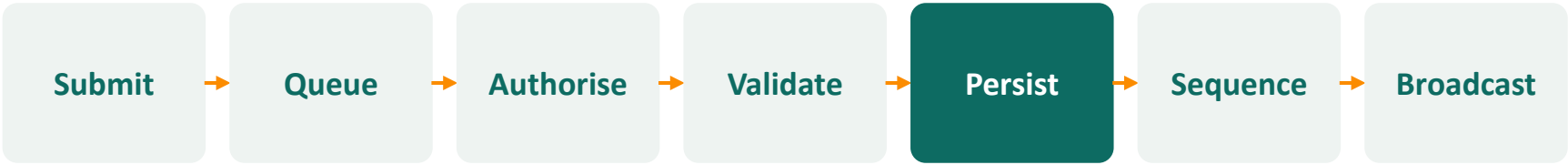
REST /api/v1

- auth: register · login · logout
- boards: list · create · get · rename · delete
- links: create · list · revoke · resolve
- members: list · change role · remove · transfer ownership
- versions: list · preview · restore (Idempotency-Key)
- health check

Socket.IO /board

- client → server: board:join · op:submit · board:leave
- server → room: op:applied · presence:update · board:state
- role:changed · access:revoked · version:created/restored · board:renamed
- JWT in the handshake · 30 ops/s · 64 KiB per message

How one change travels



Server (one FIFO queue per board): every change is stored in MongoDB before it is acknowledged or broadcast.

Client sync module (shared/src/syncClient.js)

- confirmed + pending: own changes appear instantly and roll back if rejected
- strict sequence order; duplicates ignored; a gap > 1 s or 100 buffered events → full resync
- same property edited twice → higher sequence number wins on every client

Database design and persistence

Collection	Holds	Key point
users	e-mail, bcrypt hash, display name	unique e-mail; never a password
boards	title, owner, seq, contentBytes	journal for clear and restore
boardObjects	one document per object	$jsonSchema validator in MongoDB
memberships	board, user, role	Owner stored on the board
shareLinks	SHA-256 token hash, role, expiry	tokens never stored
versions	full snapshot, trigger, number	≥ 20 newest kept
revokedTokens	logged-out token IDs	TTL index cleans up

Testing strategy and results

110
unit tests

100
integration tests

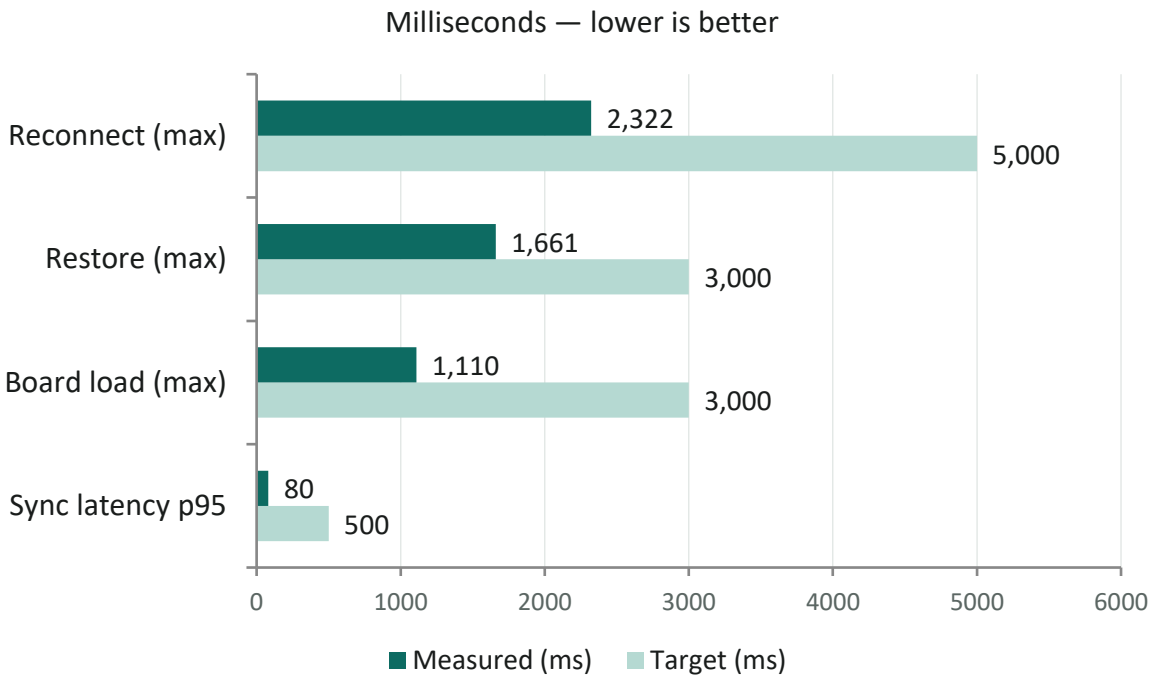
31
end-to-end tests

49/50
requirements verified

Browser (E2E, 02-11-2026)	Result
Chromium (CI) · Chrome · Edge	31 / 31 each
Playwright Firefox build	28 passed, 3 skipped
Playwright WebKit build	29 passed, 2 skipped
Stable Firefox 159 (BiDi driver)	22 passed; 7 driver limits
Apple Safari	not tested

- Levels: unit → integration with a real MongoDB and scripted clients → browser E2E
- Fault injection: server killed during clear and restore
- CI runs lint, tests, build and E2E on every Pull Request

Performance against the targets



All four targets met

- 10 clients, 10,566 samples for latency (TC-019)
- Worst browser values shown (Chrome)
- Measured 12-10-2026 on one laptop, local network only

Challenges and how we solved them

A crash in the middle of a restore	Journal on the board document + Idempotency-Key; recovery at start-up (TC-024)
A bad merge broke main (PR #28)	Repaired in PR #31; lesson: protect main and require CI
Firefox test could not simulate offline	Diagnosed: emulation leaves WebSockets open; tests corrected with evidence
Top bar collapsed at 760–1000 px	Found by a layout probe; icon buttons below 1024 px (#23)
crypto.randomUUID missing on plain HTTP	Fallback ID generator for drawing and restore (#15, #25)

Four sprints

1 — 2 — 3 — 4

Sprint 1

31 Aug – 13 Sep

Project foundation

Setup, frontend & canvas, shared errors

3 issues

Sprint 2

14 – 27 Sep

Core whiteboard

Auth, boards, sharing, tools, export, API integration, responsive layout

7 issues

Sprint 3

28 Sep – 11 Oct

Real-time collaboration

Socket.IO sync, connection lifecycle

2 issues

Sprint 4

12 – 25 Oct

Testing & finalisation

E2E, backend tests, DB integrity, frontend validation, multi-client sync, consistency, cross-browser, release

8 issues

All 20 issues and 4 milestones closed · code merged through Pull Requests #2–#33 (26 Oct – 2 Nov) with CI.

Limitations and future work

Known limitations

- HTTPS deployment not verified (TC-109 Blocked)
- Apple Safari not tested
- Single server; 15 MiB per board
- Unconfirmed changes discarded after reconnect
- Export & rename: scope decision pending

Future work

- Deploy behind HTTPS and run TC-109
- Resubmit unconfirmed changes using clientOpId
- Several servers with a Socket.IO adapter
- Undo/redo, live cursors, PDF export
- CRDT merging for text

Thank you — questions?

- Live, consistent collaboration with history and safe sharing
- 49 of 50 requirements verified · 241 automated tests · CI on every change
- Repository: github.com/RachitSMane/SE-Mini-Project-Team-11